OORA Backend Engineer Test

Nov 10, 2026 · @Syns

Design and build a staff access system for an example company: about **4 hours** in total, one short document (about 1 page) plus a small code repository, in English, back within 3 days. We expect you to build with AI. What we want to see is how you manage a backend built that way: how you design it, keep it secure, run its servers, and keep the AI working within your rules.

Imagine this project: a company opening a new skin clinic hires you as its first backend engineer. Your first project is its **staff portal**: who can sign in, and what each person can see and do. The company is an invented example: its numbers and details are made up for this test, and you give it its name.

Building with AI. Use AI tools for any part of this test, design and code, as you would at work. We score your management of the result: the rules you give the AI, what you check before anything goes live, and whether you understand every part. In the follow-up interview (about 45 minutes, live, no AI) you will walk us through your design, explain your code, and change one part of it while we watch. Anything you cannot explain counts as not answered.

The example company (you will name it)

A company opening a skin clinic in Jakarta: facials, laser, skin care and medical-grade treatments by a doctor. One clinic at opening, with a plan to grow to 10 branches.

How a customer moves through the business

1. **Lead** — sees an ad or a creator post, or is referred by a friend, then messages on WhatsApp, Instagram or the website chat.
2. **Booking** — books online, or an agent books them in chat. They get a confirmation and reminders.
3. **Visit** — the front desk checks them in, a consultant assesses them, a therapist or the doctor treats them. Rooms and equipment are limited, so the schedule matters.
4. **Payment** — a single session or a package of sessions; deposits and store credit exist. Every sale goes into a sales ledger that Finance reconciles.
5. **After the visit** — follow-ups, packages to finish, call-backs for no-shows, reviews, and referral rewards for partners.
6. **Reports** — management watches leads, bookings, show-ups, revenue, ad cost per booking and response times every day.

The people: Management, Growth, Marketing, Finance, HR, Partnerships and CRM & Sales agents at head office; Clinic Operations, Front Desk, Beauty Consultants and doctors at the clinic.

What the system holds: customer records (name, phone, visits, treatments, skin profile and allergies, photos), bookings, customer chats, the sales ledger, stock, staff attendance, and data from outside vendors.

Tools the company plans to use: a team chat and approvals app, spreadsheets, a WhatsApp Business vendor, and AI assistants for staff. Much of the software will be written with AI and reviewed by people.

Rules to follow: Indonesia's Personal Data Protection law (UU PDP); health and personal data only to those who need it; money figures only to roles that need them; no customer data in source code or URLs.

Warm-up — Name the company (3 lines)

Give the company a name and a one-line tagline, and say in one sentence why they would work for its customers in Jakarta.

Question 1 — Design the staff access system (about 1 page, a few sentences per part)

The company needs to decide how staff sign in to its internal system and what each person can open. Design it for the example company above.

Example staff structure (illustrative numbers; design for one clinic now and 10 branches later)

Department	People	Where	Works with
Management	4	Head office	Everything
Growth (product)	2	Head office	Everything
Marketing	7	Head office	Campaigns, leads, ad results
Finance	3	Head office	Sales ledger, revenue, reconciliation
HR	2	Head office	Staff directory, attendance
Partnerships	3	Head office	Creator referrals and their rewards
CRM & Sales	9	Head office	Customer chats, bookings, leads, follow-ups

Department	People	Where	Works with
Clinic Operations	2	Clinic	Patients, schedule, stock, clinic revenue
Front Desk	6	Clinic	Check-in, bookings, entering sales
Beauty Consultants	4	Clinic	Consultations, patient cards
Doctor	2	Clinic	Medical record, patient cards

Some people need one page from another department (a marketer who needs the follow-up list; a clinic lead who may see that clinic's revenue). Staff will also reach company data through AI assistants, and managers approve some actions in the team chat app.

Rules management has decided (your design must hold them):

- **Management and Growth see everything**, money included.
- **Doctors never see revenue** or any money figure, whatever access they are given.
- **Regular staff never see sales revenue.** Only Management, Growth, Finance and specifically named people may.
- The **Front Desk enters sales** but never sees the totals.
- Every exception is given to a named person, recorded with who gave it and why, and can be taken away at once.

Answer each part in a few sentences; one line per cell in the table is enough. At the end, name the **one mistake** you would most want to avoid in the whole design.

1. **Sign-in (auth):** how staff sign in (password, 2FA, manager approval, single sign-on?), how long a session lasts, what happens on a shared front-desk computer, and what happens the minute someone leaves the company.
2. **Permissions:** how you model departments, branches, pages, "may see money", "may enter but not read", and one-off exceptions. Show the permission table for the departments above and how a manager gives or takes away access.
3. **Security settings:** what you log and audit, how you protect sessions and cookies, rate limits and lockouts, how exports of customer data are approved, the database role the app uses, and how AI assistants get only what the person may see.
4. **Architecture:** the parts, the data model (tables), where the access decision is made on every request, how other services (AI assistants, approvals in the chat app) ask it, and how it grows to 10 branches. A diagram is welcome.
5. **Tools and servers:** fill in the table below. Name the actual tool or service for each line, say how far you would go at this size (run it yourself, or a managed service), and give a rough monthly cost. Assume a small team (1–2 engineers plus AI assistants), a tight budget, and users in Indonesia. Then, in a few sentences, how you would manage the

backend day to day: code structure, database changes, background jobs, API documentation, and who can change what. Where the server lives and how you run it go in your README (Question 2).

Part	Your choice (tool or service)	Run it yourself or managed	Rough cost per month	Why
Language and framework				
Database				
Background jobs and queues				
Sign-in and identity (incl. 2FA)				
Secrets and keys				
Deploys (CI/CD)				
Logging, monitoring and alerts				

Question 2 — Build it: a working mini staff portal (about 2½ hours)

Build a small **working** web service: a mini version of the company's staff portal with access per person and per page. Use Node.js + TypeScript (or the backend language you know best) and PostgreSQL or SQLite. Use fake data only. Build it ready for 10 branches: every staff member belongs to a branch, branch staff see only their own branch, and head office sees all branches. It must start on our laptop with one command (for example `docker compose up` or `npm start`).

Pages (each belongs to a department):

Page	Owner	Shows money
/bookings	**CRM & Sales**	**no**
/patients	**Clinic Operations**	**no**
/medical-record	**Doctor**	**no**
/sales-entry	**Front Desk (enter a sale only)**	**no**

Page	Owner	Shows money
/revenue	Finance	yes

What it must do

1. **Sign-in** — `POST /login` with email and password (passwords stored only as hashes), a session cookie, and `POST /logout`.
2. **The five pages** in the table above, each returning a simple page with fake data. The server checks access on **every request** and answers 403 when the person may not open it.
3. **Your access rules** — you decide which department may open which page, and how departments are given access. Keep the rules as data (for example a table in your code), not scattered if-statements, and explain your choices in the README. They must still hold the management rules in Question 1.
4. **Grants** — an admin API to give one person one extra page or "may see money", and to take it away. Each grant records who gave it, when and why; taking it away keeps the record. A removed grant or a deactivated person is refused on the **very next request**.
5. **Audit** — every refused access is logged with who, which page and when. No passwords or customer data in the logs.
6. **Seed data** — about 10 fake staff across the departments above, so we can sign in as each and try it.
7. **Tests** for every rule, runnable with one command, and a short **README** (how to run it, what you would build next).
8. **AI instruction file** — include the file your AI coding tools worked from (for example `CLAUDE.md`, `AGENTS.md` or a skill file). If you used none, write one now for the next engineer who joins: what the project is, how to run and test it, the rules it must keep, and what an AI must never do here.

Send a git repository link or a zip. Small, readable and tested beats big.

Optional, about 30 minutes — frontend (we mainly look at the backend): a simple admin screen to see one person's access and give or take away a page. Either build a plain version on top of your API, or describe in a few sentences how you would design it.

Also in your README — plan only, no need to build it: how you would run this for real, a few sentences each.

- **Where the server lives** — which provider and region, and why (cost, speed for users in Indonesia, the PDP law).

- **Customer data** — how you would store and protect patients' personal and medical information and photos: where it lives, who can read it, encryption, backups, how long it is kept.
- **Company data** — how you would keep sales, revenue and staff records apart from customer data, and who can see them.
- **Running it** — how a change reaches production safely (including database changes and rollback); who may access the servers and the database, and what happens when they leave; how you prove a backup can really be restored; and what you do in the first hour if the booking page becomes very slow on a busy Saturday.
- **Getting there** — the first steps you would take to move from your test project to a live system.

Question 3 — AI: how you use it and how you would manage it (about ½ page)

How do you use AI tools in your own work today? Which ones, for what, and what do you always check yourself before anything goes live? Name one thing the AI got wrong in this test, how you caught it, and what you changed so it would not happen again.

The company's staff and engineers will share many AI instruction and skill files (Markdown files that tell AI tools how to do a task). How would you organise them, keep them current, and review changes? What must never go in them?

We value clear reasoning and real trade-offs over long lists. If something is unclear, write down your assumption and continue.